ASSIGNMENT 5 - CO5

PART A - DESIGN

TASK A2 - DESIGN THE SYSTEM ADMINISTRATION

5 Marks | Bloom's Level: K5 - Evaluate

Case Study - University Result Portal

The university result portal is a Python web application with a database. It serves a few hundred users on normal days, but 150,000 students log in within two hours on result day. The existing deployment runs on one physical Linux server. A2 requires a complete administration and networking design: user accounts and groups, systemd services and logs, IP addressing and DNS, Nginx reverse proxy, firewall policy, SSH keys and routine automation. The alternative analysis compares hand-written shell scripts with Ansible.

The average login arrival over the two-hour window is 150,000 / 7,200 = 20.83 logins per second. This average does not represent peak bursts or total HTTP/database request rate; each login can generate multiple requests. The design must therefore include monitoring, controlled administration and a path to add application hosts as demand grows.

1. A2.1 - Users, Services and Logs (2 Marks)

1.1 User accounts and groups

Use named human accounts for people and separate non-login system identities for services. Do not share administrator credentials. Grant only the permissions required by each operational role.

Identity / group	Membership and permissions	Purpose
portal-admins	Named Linux administrators; approved sudo rules and SSH key access.	System configuration, incident response, patching and service administration.
portal-deploy	Approved release operators; write access to staging/release paths and a restricted service-restart privilege.	Deploy application releases without unrestricted root access.
portal-logreaders	Read-only access to selected portal and Nginx logs.	Support and audit staff can investigate requests without modifying logs.
portal	System account; shell set to /usr/sbin/nologin; no interactive SSH.	Runs the Python/Gunicorn application process.
postgres	Separate database service identity, controlled by PostgreSQL package/service policy.	Owns database processes and database files; not shared with the web application.

Suggested filesystem ownership: application releases are controlled by the deployment administrator and readable by the portal service; runtime state is owned by portal; the environment file is root-owned and readable only by root and the portal group (mode 0640); database files remain owned by postgres. Use distinct service identities so compromise of the web process does not grant database or administrator privileges.

Illustrative account/group commands are included in the configuration notes. Human account creation should follow university identity policy, with individual accounts, multi-factor access where available and periodic access review.

1.2 systemd services

Run the Python application under systemd as the unprivileged portal account. The supplied result-portal.service unit specifies a working directory, environment file, Gunicorn worker command, automatic restart on failure and basic systemd sandboxing. PostgreSQL and Nginx remain separate systemd services.

Service	Responsibility	Operational checks
nginx.service	Accepts client HTTP/HTTPS connections, redirects HTTP to HTTPS and proxies requests to the application.	systemctl status nginx; nginx -t; access/error log review.
result-portal.service	Runs Gunicorn and the Python application on a loopback/private listener.	systemctl status result-portal; systemctl restart/reload as appropriate; service log review.
postgresql.service	Runs the database engine and stores result records.	systemctl status postgresql; database health, connection count and backup verification.

The application service should use Restart=on-failure with a short restart delay, a controlled shutdown timeout and a non-root identity. Store credentials in a root-protected environment file or approved secret manager, not in the service unit or source code. Verify the actual WSGI import path and worker count before enabling the service.

1.3 Log monitoring and retention

Send application standard output and error to the systemd journal. Use journalctl to inspect the portal service and Nginx access/error logs for request-level analysis. Enable persistent journaling with a bounded disk quota and retention period, and use logrotate for file-based application logs. Nginx package installations commonly include their own logrotate configuration; retain a separate policy only for additional portal log files.

```
# Live application service log
sudo journalctl -u result-portal.service -f

# Errors from the current boot
sudo journalctl -p err -b

# Nginx access and error logs
sudo tail -f /var/log/nginx/result-portal-access.log
sudo tail -f /var/log/nginx/result-portal-error.log
```

Monitor HTTP 5xx rates, failed authentication, application restarts, disk usage, database connection saturation and backup completion. Logs should be access-controlled, time-synchronised and retained for a defined operational/audit period; redact passwords, session tokens and personal data.

2. A2.2 - Networking Services (2 Marks)

2.1 IP addressing plan

Use private RFC1918 addressing for internal tiers and a public NAT or load-balancer address for the public hostname. The following subnets are illustrative only and must be replaced if they overlap the university's existing network.

Network / VLAN	Subnet and gateway	Assigned roles
Management / VPN	10.20.10.0/24; gateway 10.20.10.1	Admin bastion 10.20.10.10; only authorised management clients.
DMZ / reverse proxy	10.20.20.0/24; gateway 10.20.20.1	Nginx edge 10.20.20.10; public HTTP/HTTPS termination.
Application	10.20.30.0/24; gateway 10.20.30.1	Python application hosts 10.20.30.11-10.20.30.30.
Database	10.20.40.0/24; gateway 10.20.40.1	Database primary 10.20.40.10; backup/standby 10.20.40.11.

For the initial single-host demonstration, Nginx can proxy to Gunicorn on 127.0.0.1:8000 and PostgreSQL can listen on a local Unix socket or loopback only. When the application is separated onto multiple hosts, Nginx proxies across the private application VLAN and the database accepts connections only from approved application addresses.

2.2 DNS design

Record	Target / purpose
Public A record: results.university.example	Public NAT or load-balancer IPv4 address; replace the reserved example name and address with the university's real values.
Public AAAA record	Create only if the service has a working public IPv6 address and firewall policy.

Record	Target / purpose
Internal portal-app.service.internal	Private application VIP or application host address.
Internal portal-db.service.internal	Private database address 10.20.40.10.

Public DNS should expose only the front-end address. Internal DNS resolves private application/database names for server-to-server communication. Use a low TTL during planned migrations and a normal operational TTL once the address is stable.

2.3 Nginx reverse proxy

Nginx is the internet-facing HTTP entry point. It redirects TCP/80 requests to HTTPS, terminates TLS on TCP/443, forwards the original host and client-address headers, and proxies requests to Gunicorn. The included Nginx configuration uses a loopback upstream for a single-host deployment and placeholder certificate paths. For multiple application hosts, change the upstream to the private application addresses.

```
upstream result_portal_backend {
    server 127.0.0.1:8000;
    keepalive 32;
}

server {
    listen 80;
    server_name results.university.example;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name results.university.example;
    ssl_certificate     /etc/letsencrypt/live/results.university.example/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/results.university.example/privkey.pem;

    location / {
        proxy_pass http://result_portal_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Do not expose the Gunicorn listener or database port directly to the internet. Obtain and install a valid certificate, replace the example hostname and validate the configuration with nginx -t before reloading Nginx.

2.4 Firewall policy

Source	Destination / port	Policy
Public clients	Nginx TCP/443	Allow HTTPS.
Public clients	Nginx TCP/80	Allow only to redirect to HTTPS.
Admin VPN / bastion CIDR	Server TCP/22	Allow SSH only from approved management addresses.
Nginx tier	Application TCP/8000	Allow only over the private application network.
Application tier	Database TCP/5432	Allow only from approved application hosts.
Public clients	Application TCP/8000 or database TCP/5432	Deny.

Set UFW default incoming policy to deny and outgoing policy to allow only the required destinations where egress controls are available. Add the management SSH rule before enabling the firewall. The included ufw_setup.sh defaults to a dry run; applying it requires explicit APPLY=1 and an administrator CIDR. Never enable a new firewall remotely until the allowed SSH source and a recovery path are verified.

2.5 SSH key access

Administrators generate an Ed25519 key pair on their own workstation and copy only the public key to the named server account. The private key remains on the workstation and should be protected by a passphrase. After a second key-authenticated session has been tested, disable password authentication, prohibit root login, restrict SSH to portal-admins and validate sshd configuration before reload.

```
ssh-keygen -t ed25519 -C "portal-admin"
ssh-copy-id adminuser@SERVER_IP
sudo sshd -t
sudo systemctl reload ssh
```

A2.1 - Operational validation and evidence

After creating the service identity and groups, verify account ownership, unit syntax and service state. The named human administrator must be added according to university policy; the helper script creates only the service account and groups.

```
sudo ./create_portal_accounts.sh
id portal
getent group portal-admins
sudo systemd-analyze verify /etc/systemd/system/result-portal.service
sudo systemctl daemon-reload
sudo systemctl status result-portal nginx postgresql
```

Service and log monitoring checklist

Application availability	systemctl is-active result-portal nginx postgresql; alert on repeated restarts or failed units.
Web layer	nginx -t; review Nginx access/error logs and the rate of HTTP 5xx responses.
Host capacity	df -h, free -h and uptime; alert when filesystems cross the configured disk threshold.
SSH/security	journalctl -u ssh; review repeated authentication failures and unexpected administrator access.
Database backup	Check the newest dump timestamp and perform scheduled restore tests on a separate location.

Operational review should correlate application errors with CPU/memory pressure, database connections, disk capacity and recent deployments. A backup job is successful only when its exit status is checked and a restore test proves that the dump is usable.

3. A2.3 - Automation with Shell Scripts and Cron (1 Mark)

Automate repeated work so that it occurs consistently and does not rely on an administrator remembering each step. The supplied package includes a PostgreSQL backup example, a disk-usage alert script, a cron schedule and a logrotate policy.

Routine task	Automation method	Schedule / validation
Database backup	backup_postgres.sh uses pg_dump custom format, timestamped files and a 14-day local retention rule.	Daily at 02:15; verify exit status and restore-test backups. Copy a second encrypted copy off-host.
Disk-space alerts	disk_space_alert.sh checks filesystem usage with df/awk, writes a syslog warning and optionally emails the operations team.	Every 10 minutes; threshold defaults to 80%; verify logger/mail delivery.
Log rotation	logrotate-result-portal rotates, compresses and retains application log files.	Daily via the distribution's logrotate schedule; verify rotation and permissions.
Certificate expiry / service health	A scheduled check can inspect certificate expiry and systemctl status for Nginx, portal and PostgreSQL.	Daily certificate check; frequent service health checks or monitoring alerts.

Example /etc/cron.d/result-portal entries:

```
# Daily PostgreSQL backup at 02:15 UTC
15 2 * * * postgres /usr/local/sbin/backup_postgres.sh

# Disk usage check every 10 minutes
*/10 * * * * root /usr/local/sbin/disk_space_alert.sh
```

Logrotate is normally invoked by the operating system's daily schedule; the included policy defines how result-portal log files are rotated, compressed and retained. A backup is not complete until a restore test succeeds. A local backup on the same physical disk does not protect against disk/server loss; retain a separate off-host copy.

4. Alternative Concept - Ansible Instead of Hand-Written Shell Scripts

Ansible expresses the desired configuration as inventory, variables and playbooks. It connects to managed Linux servers over SSH and uses modules to ensure packages, groups, accounts, directories and services reach the declared state. Many built-in modules are idempotent, meaning rerunning a task that already achieved its desired state does not make another change. Arbitrary shell commands are not automatically idempotent and still need careful design.

Evaluation dimension	Hand-written shell scripts	Ansible
One-server setup	Lower initial overhead; scripts can be written and tested quickly for one known host.	Higher initial overhead to learn inventory, YAML, playbooks, variables and test the controller/SSH path.
Twenty-server setup	Scripts can be looped over hosts, but inventory, per-host exceptions, retries and drift checks must be added and maintained.	One playbook and inventory can configure all hosts consistently; parallel forks shorten rollout time.
Configuration drift	Risk increases when hosts are changed manually or scripts are not rerun; scripts often encode procedural steps.	Desired-state modules reduce drift when playbooks are rerun; exceptions and non-idempotent tasks still require controls.
Learning and maintenance	Bash is familiar and flexible; quoting, error handling, host loops and rollback logic become harder as scope grows.	Moderate learning effort; structured variables/roles and reusable modules improve standardisation but add tool/inventory maintenance.

4.1 Time estimate for one and twenty servers

The question does not provide measured timings, so the following is a transparent planning model, not an experimental result. It compares the same baseline tasks: create the service identity/groups, install packages, deploy configuration, set network/SSH policy and validate the services.

Approach	Initial authoring / test effort	Per-server application and verification	1 server	20 servers
Shell scripts	45 min	8 min	45 + 8 = 53 min	45 + (20 x 8) = 205 min (3 h 25 min)
Ansible	75 min	3 min	75 + 3 = 78 min	75 + (20 x 3) = 135 min (2 h 15 min)

For one server, the shell-script approach is estimated to be 25 minutes faster because the one-time Ansible learning and playbook setup is not amortised. For twenty servers, Ansible is estimated to save 70 minutes because the same desired-state tasks are applied and checked across the inventory. These figures are assumptions for comparison; actual timings depend on packages, network, authentication, server differences, testing and failure recovery.

4.2 Complexity and operational risk

Let H be the number of managed hosts and T the number of configuration tasks. Both approaches still perform approximately O(H x T) underlying host-task operations. A script-based approach that processes hosts sequentially has a rollout time proportional to H x T. Ansible also has O(H x T) total task work, but with F parallel forks the approximate rollout wall time can approach O(ceil(H/F) x T), subject to network and service dependencies. Both require O(H + T) configuration/inventory storage; Ansible adds structured playbook, variable and inventory data.

Ansible does not remove the need for testing: a task implemented with a raw shell command can still be non-idempotent. Use check mode and diff output where supported, validate variables and host groups, protect secrets with an approved vault/secret manager, and keep a controlled rollback plan. For one machine, shell scripts may be simpler; for twenty similar machines, Ansible is preferable because repeatability and drift control outweigh its higher initial learning cost.

5. Implementation and Validation Procedure

A2 does not ask for a C program or a Complete-the-Code implementation. The supplied Bash scripts and service/network configuration files are practical examples supporting the requested design. They are not automatically deployed during preparation.

File	What to validate before use
result-portal.service	Application path, Python environment, WSGI import path, environment file, worker count and PostgreSQL dependency.
nginx-result-portal.conf	Actual DNS name, installed certificate paths, app upstream and request-size/timeouts.
ssh-hardening.conf	Approved admin group exists and a tested public-key login works before password access is disabled.
ufw_setup.sh	Correct administrator VPN CIDR; inspect dry-run output before APPLY=1. Keep console/recovery access.
backup_postgres.sh	Database type/name, peer authentication, backup directory permissions, retention, off-host copy and restore test.
disk_space_alert.sh / cron	Threshold, email/syslog configuration, executable path and cron service status.
ansible/portal_admin.yml	Inventory addresses, privilege escalation, package source, variables, templates, secret handling and application readiness.

Useful validation commands after installing the corresponding files:

```
sudo systemd-analyze verify /etc/systemd/system/result-portal.service
sudo systemctl daemon-reload
sudo systemctl status result-portal nginx postgresql
sudo nginx -t
sudo sshd -t
sudo ufw status verbose
sudo journalctl -u result-portal.service --since today
sudo logrotate -d /etc/logrotate.d/result-portal
```

For Ansible, first validate the YAML/playbook against a disposable test host, then run `ansible-playbook --check --diff` before applying changes. The provided example intentionally does not deploy application secrets or enable

the app service before the application code, environment file and TLS certificate exist.

6. Conclusion

The proposed administration plan separates human administrators from application and database service identities, uses systemd for controlled service lifecycle, collects persistent journal and Nginx logs, and applies least-privilege network exposure through Nginx, UFW and SSH keys. Cron and logrotate automate backups, disk alerts and retention. The Ansible alternative costs more to learn and prepare for one host, but its reusable desired-state playbooks improve consistency and reduce rollout time across twenty servers. The address ranges, DNS names and time calculations are explicit planning examples and must be replaced or measured in the university environment.

7. Technical References

[1] Ubuntu Server Documentation, *OpenSSH Server* - Ed25519 keys, authorized_keys and sshd configuration. https://ubuntu.com/server/docs/openssh-server/

[2] Ubuntu Server Documentation, *Firewall / UFW* - allow/deny rules, source CIDRs and dry-run operation. https://ubuntu.com/server/docs/firewalls/

[3] NGINX Documentation, *Reverse Proxy* - proxy_pass, upstream servers and forwarded headers. https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/

[4] Ansible Core Documentation, *Playbooks* - desired state, idempotency and running playbooks. https://docs.ansible.com/projects/ansible-core/devel/playbook_guide/playbooks_intro.html

[5] systemd Documentation, *journald.conf* and *systemd-journald.service* - persistent journal storage and retention limits. https://www.freedesktop.org/software/systemd/man/latest/journald.conf.html ; https://freedesktop.org/software/systemd/man/latest/systemd-journald.html

Source alignment: Assignment 5 - CO5, Part A - Design, Task A2 - Design the System Administration, 5 Marks, page 44 of the supplied question paper. General task requirements are on page 42.